4

Armazenamento

Universidade Estadual da Paraíba (UEPB)
Paradigmas de Programação
Kláudio Henrique Mascarenhas Medeiros

Armazenamento

- Em linguagens imperativas, as variáveis são objetos que contém um valor que pode ser inspecionado e atualizado.

Armazenamento

- Em linguagens imperativas, as variáveis são objetos que contém um valor que pode ser inspecionado e atualizado.

- Em linguagens declarativas, as variáveis são mais parecidas com variáveis matemáticas.

Armazenamento

- Em linguagens imperativas, as variáveis são objetos que contém um valor que pode ser inspecionado e atualizado.

- Em linguagens declarativas, as variáveis são mais parecidas com variáveis matemáticas.

- Essas variáveis podem ser utilizadas para modelar objetos ou valores do mundo real.

Variáveis temporárias ou persistentes

- As variáveis temporárias são criadas e utilizadas dentro de um programa particular. Podem ser atualizadas por atribuição.

Variáveis temporárias ou persistentes

- As **variáveis temporárias** são criadas e utilizadas dentro de um programa particular. Podem ser atualizadas por atribuição.

- As **variáveis persistentes** existem independente de um programa particular. Podem ser atualizadas de forma seletiva (e.g. arquivo, BDs).

Variáveis (memória)

- Podemos ver uma variável como um modelo abstrato de armazenamento.

Variáveis (memória)

- Podemos ver uma variável como um **modelo abstrato de armazenamento**.

- Podemos ver a memória como sendo uma coleção de células, de modo que cada célula pode ter um **estado**: alocada ou desalocada.

Variáveis (memória)

- Podemos ver uma variável como um **modelo abstrato de armazenamento**.

- Podemos ver a memória como sendo uma coleção de células, de modo que cada célula pode ter um **estado**: alocada ou desalocada.

- Cada célula pode ter um valor, armazenável ou indefinido.

Variáveis (memória)

Muitas vezes, variáveis podem ter nome, sendo entendidas como sendo um espaço de memória nomeado

Variáveis (memória)

Em C, o uso de alocação dinâmica nos permite definir variáveis anônimas

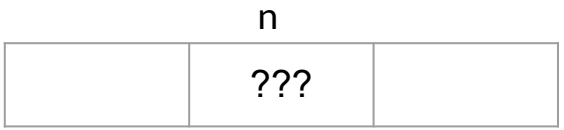
Variáveis (memória)

- Nesse caso, a memória está desalocada.

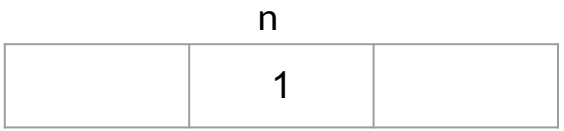
- Nesse caso, n está alocada, mas com valor indefinido.

- Nesse caso, n está alocada e com valor definido.

int n;



n = n + 1;



Variáveis (memória)

- Os **valores armazenáveis** são os valores que podem ser armazenados em células simples.

Variáveis (memória)

- Os **valores armazenáveis** são os valores que podem ser armazenados em células simples.

- Em linguagens como C++, valores primitivos e apontadores (ponteiros).

Variáveis (memória)

- Os **valores armazenáveis** são os valores que podem ser armazenados em células simples.

- Em linguagens como C++, valores primitivos e apontadores (ponteiros).

- Em linguagens como Java, valores primitivos e ponteiros para objetos. Os objetos em si não são armazenáveis, mas são acessados, implicitamente.

Variáveis (memória)

- As **variáveis simples** são variáveis que podem ter um valor armazenável. As **variáveis compostas** são variáveis de um tipo composto, ocupando várias células.

Variáveis (memória)

- As **variáveis simples** são variáveis que podem ter um valor armazenável. As **variáveis compostas** são variáveis de um tipo composto, ocupando várias células.

- É um exemplo de variável composta uma estrutura (struct) na linguagem de programação C.

Variáveis (memória)

- Exemplo: uma estrutura em C

```
struct Data{ int dia; int mes; int ano;};
...
struct Data data = {16, 2, 2023};
//printf("%d\n", &data.dia); //para obter endereço
```

16	2	2023

- Exemplo: um array de estruturas em C

```
struct Data datas[2];
...
```

datas[0].dia datas[0].mes datas[0].ano ...

16	2	23	14	…

Variáveis compostas

- As variáveis podem ser atualizadas de maneira total ou seletiva.

Variáveis compostas

- As variáveis podem ser atualizadas de maneira total ou seletiva.

- Na **atualização total**, um passo atualiza todos os componentes de uma vez (e.g. data[0] = hoje;).

Variáveis compostas

- As variáveis podem ser atualizadas de maneira total ou seletiva.

- Na **atualização total**, um passo atualiza todos os componentes de uma vez (e.g. data[0] = hoje;).

- Na **atualização seletiva**, um passo atualiza apenas um componente (e.g. data[0].dia = 16;).

Cópia e referência(memória)

- Exemplo: atribuição e tipos compostos

```
class Date { ... }
Date d1 = new Date(...);
Date d2 = d1;
```

- Em uma situação como essa, o que pode acontecer?
  - d1 e d2 apontarão para o mesmo objeto?
  - d1 e d2 possuirão os mesmos valores para os mesmos atributos?

Cópia e referência

- Quando um programa atribui um valor composto para uma variável, podem acontecer algumas destas abaixo:

Cópia e referência

- Quando um programa atribui um valor composto para uma variável, podem acontecer algumas destas abaixo:
    - os componentes são copiados do valor composto para todos os respectivos componentes da variável (semântica de cópia)

Cópia e referência

- Quando um programa atribui um valor composto para uma variável, podem acontecer algumas destas abaixo:
  - os componentes são copiados do valor composto para todos os respectivos componentes da variável (**semântica de cópia**)
  - a variável composta passa a ter uma referência (ou ponteiro) para o valor composto (**semântica de referência**).

Cópia e referência

- A linguagem Java trabalha com as duas semânticas em situações diferentes.

Cópia e referência

- A linguagem Java trabalha com as duas semânticas em situações diferentes.
    - para valores de tipos primitivos, a semântica é a de cópia.

Cópia e referência

- A linguagem Java trabalha com as duas semânticas em situações diferentes.
  - para valores de tipos primitivos, a semântica é a de cópia.
  - para valores de tipos objeto, a semântica é a de referência.

Cópia e referência

- A linguagem Java trabalha com as duas semânticas em situações diferentes.
    - para valores de tipos primitivos, a semântica é a de cópia.
    - para valores de tipos objeto, a semântica é a de referência.
- Pode-se ter semântica de cópia com objetos (clone).

Cópia e referência

```
public static void main(String[] args) {
    int num1 = 25;
    int num2 = num1;
    num1++;
    System.out.println("num1: " + num1 + ", num2: " + num2); // 26 25
    List<String> lista1 = new ArrayList<String>();
    List<String> lista2 = lista1;
    lista1.add("Klaus");
    System.out.println(lista2.size()); // 1
}
```

Cópia e referência

```
# Exemplo de semântica de referência
lista1 = [1, 2, 4]
lista2 = lista1
print(id(lista1), id(lista2)) # lista1 e 2 com mesmo id
lista1.append(8)
print(lista1, lista2)
print(id(lista1), id(lista2)) # sem mudança no id
print()

lista1 = [2, 4, 8]
lista2 = lista1.copy()
print(id(lista1), id(lista2)) # lista1 e 2 com difer. id
lista1.append(9)
print(lista1, lista2)
print(id(lista1), id(lista2)) # sem mudança no id
```

```
2612225764736 2612225764736
[1, 2, 4, 8] [1, 2, 4, 8]
2612225764736 2612225764736

2612225762240 2612225761984
[2, 4, 8, 9] [2, 4, 8]
2612225762240 2612225761984
```

Comandos

- Os **comandos** são trechos de programa utilizados para atualizar variáveis.

Comandos

- Os **comandos** são trechos de programa utilizados para atualizar variáveis.

- São característicos de linguagens de paradigma imperativo, concorrente ou orientada a objetos.

Comandos

- Os **comandos** são trechos de programa utilizados para atualizar variáveis.

- São característicos de linguagens de paradigma imperativo, concorrente ou orientada a objetos.

- Existem vários tipos de comando diferentes, listados nos próximos slides.

Comandos

- Comandos skip: não possuem efeito nenhum. Pode ter algumas utilidades (em Java, ‘;’ é um skip)

Comandos

- Comandos skip: não possuem efeito nenhum. Pode ter algumas utilidades (em Java, ';' é um skip)

- Comandos de atribuição: servem para atualização total ou seletiva de variáveis.

  - podem ser múltiplas (e.g. x := y := z := 10)
  - podem ser simultâneas (e.g. x, y = 10, 8)

Comandos

- Comandos de chamada a procedimento: aplica o procedimento a alguns argumentos (e.g. $P(pr_1, ..., pr_n)$). O pr_i pode ser uma expressão ou acesso a variável.

Comandos

- Comandos de chamada a procedimento: aplica o procedimento a alguns argumentos (e.g. $P(pr_1, ..., pr_n)$). O pr_i pode ser uma expressão ou acesso a variável.

- Esses procedimentos podem mudar os valores de variáveis passadas por argumento (e.g. via passagem por referência) ou de variáveis declaradas fora do procedimento (e.g. variável global).

Comandos

- Comandos sequenciais: indica comandos que são executados de forma sequencial (e.g. C1; C2). Toda linguagem imperativa suporta esse tipo de comando.

Comandos

- Comandos sequenciais: indica comandos que são executados de forma sequencial (e.g. C1; C2). Toda linguagem imperativa suporta esse tipo de comando.

- Comandos colaterais: indica que a ordem de execução dos comandos é irrelevante (e.g. C1, C2). Normalmente, suportada por linguagens concorrentes.

Comandos

- Comandos colaterais: indica que a ordem de execução dos comandos é irrelevante (e.g. C1, C2). Normalmente, suportada por linguagens concorrentes.
    - os comandos colaterais são não-determinísticos, no sentido da sequência de comandos executada não ser previsível (mas o resultado pode ser previsível).

Comandos

- Comandos colaterais: indica que a ordem de execução dos comandos é irrelevante (e.g. C1, C2). Normalmente, suportada por linguagens concorrentes.
    - os comandos colaterais são não-determinísticos, no sentido da sequência de comandos executada não ser previsível (mas o resultado pode ser previsível).
    - caso um comando não acesse ou altere uma variável de outro comando, o comando colateral é dito ser efetivamente determinístico.

Comandos

- Comandos condicionais: possui um conjunto de subcomandos, onde um deles é escolhido, baseado em algum critério, para ser executado (e.g. if E then c1 else c2).

Comandos

- Comandos condicionais: possui um conjunto de subcomandos, onde um deles é escolhido, baseado em algum critério, para ser executado (e.g. if E then c1 else c2).

- Normalmente, a expressão tem um booleano como resultado, mas nem sempre isso acontece.

Comandos

- Comandos iterativos: possui um sub-comando que é executado de forma repetida até uma condição de término ser atendida (e.g. for i in range(1, 10):, for elem in lista: em Python, while(cond)).

Comandos

- Comandos iterativos: possui um sub-comando que é executado de forma repetida até uma condição de término ser atendida (e.g. for i in range(1, 10):, for elem in lista: em Python, while(cond)).

- As iterações podem ser:
  - definidas: sabemos o número de loops que ocorrerão antecipadamente (e.g. for)

Comandos

- Comandos iterativos: possui um sub-comando que é executado de forma repetida até uma condição de término ser atendida (e.g. for i in range(1, 10):, for elem in lista: em Python, while(cond)).

- As iterações podem ser:
  - definidas: sabemos o número de loops que ocorrerão antecipadamente (e.g. for)
  - indefinidas: não sabemos desse número de loops (e.g. while, do..while, repeat..until)

Expressões com efeitos colaterais

- De modo geral, expressões são utilizadas para fazerem algo e resultarem em um valor.

Expressões com efeitos colaterais

- De modo geral, expressões são utilizadas para fazerem algo e resultarem em um valor.

- Caso a expressão faça mais que isso, é dito que ela tem efeito colateral.

Expressões com efeitos colaterais

- De modo geral, expressões são utilizadas para fazerem algo e resultarem em um valor.

- Caso a expressão faça mais que isso, é dito que ela tem efeito colateral.

- É um exemplo de efeito colateral muito comum (e interessante) a alteração do valor de uma variável.

Expressões com efeitos colaterais

As linguagens funcionais são muitas vezes caracterizadas por não possuírem efeitos colaterais em expressões e funções

Referências

SEBESTA, Robert M. **Conceitos de Linguagens de Programação**. Bookman, 2018.